

Demystifying Agentic AI

Tool Calling, Skills, and Modern Architecture

Kyle Pounder Toby Shearman

IAB Fall Meeting
University of Arizona

The task

“The test suite is failing. Fix it.”

What `run_tests()` shows (excerpt):

```
def test_median_even():
>     assert median([4, 1, 3, 2]) == 2.5
E     assert 3 == 2.5
E         + where 3 = median([4, 1, 3, 2])

FAILED test_stats.py::test_median_even
1 failed, 2 passed in 0.01s
```

The code under test, `stats.py`:

```
def mean(xs):
    return sum(xs) / len(xs)

def median(xs):
    xs = sorted(xs)
    n = len(xs)
    return xs[n // 2]
```

A coding agent at work

> The test suite is failing. Fix it.

- I'll start by running the tests.

- `run_tests()`

```
FAILED test_stats.py::test_median_even
```

```
assert 3 == 2.5 (median([4, 1, 3, 2]) is 3)
```

```
1 failed, 2 passed
```

A coding agent at work

✓ `run_tests()` 1 failed, 2 passed

- The failure is in `median`. Let me read it.

- `read_file("stats.py")`

```
def median(xs):  
    xs = sorted(xs)  
    n = len(xs)  
    return xs[n // 2]
```

A coding agent at work

✓ `run_tests()` 1 failed, 2 passed

✓ `read_file("stats.py")`

- For even lengths it should average the middle two.

- `edit_file(`

- `path: "stats.py",`

- `old_text: "return xs[n//2]",`

- `new_text: "if n % 2 == 0:`

- `return (xs[n // 2 - 1] + xs[n // 2]) / 2`

- `return xs[n // 2]"`

`error: old_text not found in stats.py`

A coding agent at work

✓ `run_tests()` 1 failed, 2 passed

✓ `read_file("stats.py")`

✗ `edit_file(...)` error: old_text not found

- The spacing didn't match. Retrying with the exact text.

- `edit_file(`

- `path: "stats.py",`

- `old_text: "return xs[n // 2]",`

- `new_text: "if n % 2 == 0: ... return xs[n // 2]"`

- `ok`

A coding agent at work

```
✓ run_tests() 1 failed, 2 passed
✓ read_file("stats.py")
✗ edit_file(...) error: old_text not found
✓ edit_file(...) ok
```

- `run_tests()`
3 passed
- Fixed: median now averages the two middle values for even-length lists. All 3 tests pass.

That looks like magic. What is going on?

An LLM

reads the history
and chooses
the next action

Three tools

read_file,
run_tests, edit_file

A loop

runs the action,
adds the result to
the history, repeats

Next: each piece, with the math that pins it down.

What is a tool?

$$\tau = \left(\underbrace{\text{name}, S}_{\text{interface} \text{ — sent to the model}}, \underbrace{f}_{\text{implementation} \text{ — host-only}} \right)$$

- The **interface** (name, S) is what the model reads and calls against; S is the *schema* (a JSON Schema)
- The implementation $f : S \rightarrow \mathcal{Y}$ is what runs, on the host, never sent to the model
- $\mathcal{X}$: all possible arguments ($S \subseteq \mathcal{X}$, the valid ones). $\mathcal{Y}$: all possible results.

The model writes down a call from the interface alone. The host checks $x \in S$, then runs f . If $x \notin S$, it returns an error message instead.

The literature calls this a “function interface to an external program” (Wang et al., 2024).

Let's fill it in: $\tau = \text{edit_file}$

In plain English: “`edit_file(path, old_text, new_text)` replaces `old_text` with `new_text` in the file at `path`. If it can't, it says why.”

What would you put in each blank? Call it out before the click.

S ?

f ?

$\mathcal{Y}$?

Let's fill it in: $\tau = \text{edit_file}$

In plain English: “`edit_file(path, old_text, new_text)` replaces `old_text` with `new_text` in the file at `path`. If it can't, it says why.”

What would you put in each blank? Call it out before the click.

S objects with exactly the keys `path`, `old_text`, `new_text`

f ?

$\mathcal{Y}$?

Let's fill it in: $\tau = \text{edit_file}$

In plain English: “`edit_file(path, old_text, new_text)` replaces `old_text` with `new_text` in the file at `path`. If it can't, it says why.”

What would you put in each blank? Call it out before the click.

S objects with exactly the keys `path`, `old_text`, `new_text`

f replace `old_text` by `new_text` in the file; return “ok” or an error message

$\mathcal{Y}$
?

Let's fill it in: $\tau = \text{edit_file}$

In plain English: “`edit_file(path, old_text, new_text)` replaces `old_text` with `new_text` in the file at `path`. If it can't, it says why.”

What would you put in each blank? Call it out before the click.

S objects with exactly the keys `path`, `old_text`, `new_text`

f replace `old_text` by `new_text` in the file; return “ok” or an error message

$\mathcal{V}$ text

Tools in the example

What does each tool return? Guess, then click.

name	<i>S</i> : allowed arguments	<i>f</i> : what the host does	result in $\mathcal{Y}$
read_file	a path (string)	reads the file	?
run_tests	none, or a path	runs the test suite	?
edit_file	a path, the old text, the new text	replaces old with new	?

Tools in the example

What does each tool return? Guess, then click.

name	<i>S</i> : allowed arguments	<i>f</i> : what the host does	result in $\mathcal{Y}$
read_file	a path (string)	reads the file	its text
run_tests	none, or a path	runs the test suite	?
edit_file	a path, the old text, the new text	replaces old with new	?

Tools in the example

What does each tool return? Guess, then click.

name	<i>S</i> : allowed arguments	<i>f</i> : what the host does	result in $\mathcal{Y}$
read_file	a path (string)	reads the file	its text
run_tests	none, or a path	runs the test suite	pass/fail summary, traceback
edit_file	a path, the old text, the new text	replaces old with new	?

Tools in the example

What does each tool return? Guess, then click.

name	S : allowed arguments	f : what the host does	result in $\mathcal{Y}$
read_file	a path (string)	reads the file	its text
run_tests	none, or a path	runs the test suite	pass/fail summary, traceback
edit_file	a path, the old text, the new text	replaces old with new	“ok” or an error message

S checks only the *shape* of the arguments. Whether `old_text` occurs in the file is checked when f runs, and a failure comes back as an ordinary result in $\mathcal{Y}$. $T = \{\tau_1, \dots, \tau_k\}$ denotes the agent's set of tools.

What the LLM sees and can do

Sees a history $h_t \in H$: the user's request, then one (action, result) pair per step so far.

$h_2 = \left(\text{"The test suite is failing. Fix it."}, \right.$
 $\left(\text{call}(\text{run_tests}, \{\}), \text{"1 failed, 2 passed"} \right),$
 $\left. \left(\text{call}(\text{read_file}, \{\text{path: "stats.py"}\}), \text{"def median(xs): ..."} \right) \right)$

Can choose an action $a_t \in A = \{\text{answer}(m)\} \cup \{\text{call}(\tau, x) : \tau \in T, x \in \mathcal{X}\}$.

What's an example of one?

H is the set of all such histories; x can be any argument object in $\mathcal{X}$.

What the LLM sees and can do

Sees a history $h_t \in H$: the user's request, then one (action, result) pair per step so far.

$h_2 = \left(\text{"The test suite is failing. Fix it."}, \right.$
 $\left(\text{call}(\text{run_tests}, \{\}), \text{"1 failed, 2 passed"} \right),$
 $\left. \left(\text{call}(\text{read_file}, \{\text{path: "stats.py"}\}), \text{"def median(xs): ..."} \right) \right)$

Can choose an action $a_t \in A = \{\text{answer}(m)\} \cup \{\text{call}(\tau, x) : \tau \in T, x \in \mathcal{X}\}.$

<code>call(edit_file, {path: "stats.py", ...})</code>	run a tool τ on arguments x
<code>call(run_tests, {})</code>	run a tool τ on arguments x
<code>answer("Fixed median; 3 tests pass.")</code>	reply to the user with text m , and stop

H is the set of all such histories; x can be any argument object in $\mathcal{X}$.

The LLM chooses

$$\pi : H \rightarrow \Delta(A), \quad a_t \sim \pi(\cdot \mid h_t)$$

$\Delta(A)$ is the set of probability distributions over A . Given the history, the LLM puts a probability on every possible next action, and one is drawn at random ($\sim$).

$\pi(\cdot \mid h_2)$ for our example (made-up numbers)

`call(edit_file, {...})`  0.80

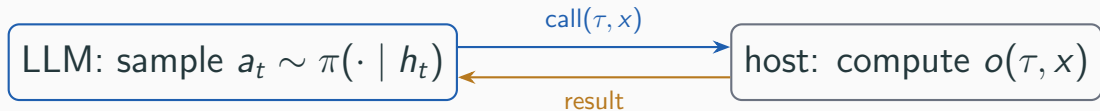
`call(run_tests, {...})`  0.10

`call(read_file, {...})`  0.07

everything else, incl. `answer(m)`  0.03

How π learns to choose well comes from training; for this talk it is a black box we sample from.

The loop



The model samples an action. If it's a call, the host checks the schema, runs the tool (or returns an error), and the result goes back to the model. If it's an answer, the loop stops.

The loop, precisely

$$o(\tau, x) = \begin{cases} f_\tau(x) & x \in S_\tau \\ \text{an error message} & \text{otherwise} \end{cases}$$

$$h_{t+1} = h_t \frown (a_t, o(\tau, x)) \quad \text{if } a_t = \text{call}(\tau, x)$$

stop when $a_t = \text{answer}(m)$ or $t = N$

$\frown$ appends one entry to the end of the history. h_0 is the user's request. N is a step budget.

The loop, on our example

h_0 = "The test suite is failing. Fix it." Each step appends one (action, result) pair. What does each step return?

t	action a_t	result $o(\tau, x)$
0	<code>call(run_tests, {})</code>	1 failed, 2 passed
1	<code>call(read_file, {path: "stats.py"})</code>	the source of stats.py
2	<code>call(edit_file, {old_text: "return xs[n//2]", ...})</code>	?
3	<code>call(edit_file, {old_text: "return xs[n // 2]", ...})</code>	ok
4	<code>call(run_tests, {})</code>	3 passed
5	<code>answer("Fixed median; 3 tests pass.")</code>	?

The loop, on our example

h_0 = "The test suite is failing. Fix it." Each step appends one (action, result) pair. What does each step return?

t	action a_t	result $o(\tau, x)$
0	<code>call(run_tests, {})</code>	1 failed, 2 passed
1	<code>call(read_file, {path: "stats.py"})</code>	the source of stats.py
2	<code>call(edit_file, {old_text: "return xs[n//2]", ...})</code>	error: old_text not found
3	<code>call(edit_file, {old_text: "return xs[n // 2]", ...})</code>	ok
4	<code>call(run_tests, {})</code>	3 passed
5	<code>answer("Fixed median; 3 tests pass.")</code>	?

Step 2's old_text looks right – what actually happens?

The loop, on our example

h_0 = "The test suite is failing. Fix it." Each step appends one (action, result) pair. What does each step return?

t	action a_t	result $o(\tau, x)$
0	<code>call(run_tests, {})</code>	1 failed, 2 passed
1	<code>call(read_file, {path: "stats.py"})</code>	the source of stats.py
2	<code>call(edit_file, {old_text: "return xs[n//2]", ...})</code>	error: old_text not found
3	<code>call(edit_file, {old_text: "return xs[n // 2]", ...})</code>	ok
4	<code>call(run_tests, {})</code>	3 passed
5	<code>answer("Fixed median; 3 tests pass.")</code>	(loop stops)

Step 5 is an answer. Does the loop take another step?

What is a skill?

A folder of instructions, plus optional scripts. It runs nothing by itself.

```
fix-failing-tests/  
  SKILL.md
```

```
---  
name: fix-failing-tests  
description: Use when asked to fix  
  failing tests or a failing build.  
---  
1. Run the full test suite first; read  
  the failure before opening any file.  
2. Read the code under test, not just  
  the test.  
3. Make the smallest change that fixes  
  the failure.  
4. Do not edit a test to make it pass  
  unless asked.  
5. Rerun the full suite, not just the  
  failing test.  
6. Report what you changed and why.
```

How the agent uses a skill

- **Session start:** the agent sees only each skill's name and one-line description.

How the agent uses a skill

- **Session start:** the agent sees only each skill's name and one-line description.
- **A task arrives** (“fix the failing tests”): the description matches, so the agent reads `SKILL.md` with an ordinary file-read call.
- **Now** the instructions are in its context. It may also run the bundled script, another ordinary tool call.

Why load on demand

Context budget. n skills, each with a description of d tokens and instructions of c tokens; k skills used:

$$\underbrace{n(d + c)}_{\text{load everything}} \quad \text{vs.} \quad \underbrace{nd + kc}_{\text{index, then load on demand}}$$

Illustrative numbers: $n = 50$, $d = 50$, $c = 2000$, $k = 1$ gives 102,500 versus 4,500 tokens.

Loading a skill is a tool call. It changes what the model conditions on, not the model itself:

$$h_{t+1} = h_t \curvearrowright (\text{call}(\text{read_file}, \text{SKILL.md}), \text{ text of SKILL.md})$$

Scoping. At each step the agent sees only $T_t \subseteq T$: fewer ways to be wrong, and least privilege.

Takeaways

1. An agent is a model in a loop: it emits structured calls, the host validates and runs them, and the results go back into the history.
2. Reliability is a design problem: precise schemas, small scoped tool sets, and verification steps (like rerunning the tests) make errors easier to catch.

Questions?